

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

E.K.K.Tharushi Nethmini Edirisinghe

(IT22083296)

BSc (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

A HUMAN-LIKE AGENT FOR SCREEN-AWARE TASK AUTOMATION

E.K.K.Tharushi Nethmini Edirisinghe

(IT22083296)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of
Science (Hons) in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

DECLARATION

I declare that this is my own work and this dissertation¹ does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature

Date



22/04/2026

.....

.....

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor

Date

.....

.....

ABSTRACT

This research presents the design and implementation of a Large Language Model (LLM)-based reasoning and prompt engineering module for the Human-Like Agent for Screen-Aware Task Automation system. The primary function of this component is to interpret user-provided natural language instructions and transform them into structured, step-by-step user interface (UI) interaction sequences suitable for automated execution.

The module employs structured prompt engineering techniques to guide the LLM in maintaining contextual awareness, resolving ambiguities in user instructions, and generating consistent, machine-readable action plans. These plans are designed to support both functional and exploratory testing scenarios within graphical user interfaces.

The system operates in conjunction with a visual perception module, which provides real-time structured representations of on-screen elements. Based on this input, the LLM dynamically adapts its action plans in response to UI changes, incomplete states, or execution failures, enabling iterative and feedback-driven task execution.

Unlike conventional automation frameworks that depend on internal application access, accessibility APIs, or predefined UI hierarchies, the proposed approach utilizes external visual observation through a camera feed. This design allows interaction with black-box systems and environments where traditional automation methods are limited or infeasible.

By integrating LLM-based reasoning with structured prompt design and iterative feedback mechanisms, the proposed module demonstrates a flexible and extensible approach to GUI automation. While the system is evaluated within controlled environments, the results indicate its potential for enabling adaptive, human-like interaction in cross-platform testing scenarios.

Keywords – Large Language Model (LLM), Prompt Engineering, GUI Automation, Black-Box Testing

ACKNOWLEDGEMENT

I would like to express my deep gratitude for all the people who helped me to continue my 4th year research project. First, I greatly appreciate our Research Project supervisor Mr. Vishan Jayasinghearachchi, who provided the guidance and encouragement to continue this project successfully. I am also grateful to our co-supervisor Ms. Kaushalya Rajapaksa for the valuable and constructive suggestions given during the planning of our research work.

I also acknowledge my Research Project leader Annesiyani Srikanthan and other group members who have been supportive in pursuing these goals. Finally, I would like to give my heartfelt gratitude to my parents and friends who helped me and provided unending inspiration.

TABLE OF CONTENTS

DECLARATION	<i>i</i>
ABSTRACT	<i>ii</i>
ACKNOWLEDGEMENT.....	<i>iii</i>
LIST OF FIGURES	<i>vi</i>
LIST OF TABLES	<i>vii</i>
LIST OF ABBREVIATIONS	<i>viii</i>
1. INTRODUCTION.....	1
1.1 General Introduction	1
1.2 Background Literature	2
1.3 Limitation in the Current Approach	3
1.4 Research Gap	4
2. RESEARCH PROBLEM.....	7
2.1 Problem Context	7
2.2 Problem Statement	7
2.3 Core Research Problem.....	8
3. RESEARCH OBJECTIVES	9
3.1 Main Objective.....	9
3.2 Specific Objective	9
4. METHODOLOGY	11
4.1 System Architecture	11
4.2 Specific Component Description	15
4.2.1 Purpose	15
4.2.2 Workflow Overview	16
4.2.3 Input Processing.....	17
4.2.4 Prompt Engineering	17
4.2.5 Instruction Parsing and Action Plan Generation	17
4.2.6 Integration with Visual Perception Module	17
4.2.7 Execution and Error Handling	18
4.2.8 Exploratory Branching.....	18
4.2.9 Optimization and Evaluation.....	18
4.3 Prompt Engineering and LLM Action Planning Framework	18
4.3.1 LLM Input Formulation	19
4.3.2 Prompt Template Strategy	19

4.3.3	Reasoning and Task Decomposition	19
4.3.4	Structured and Action Plan	19
4.3.5	Output Validation and Correction.....	19
4.3.6	Feedback Aware Re-prompting	20
4.3.7	LLM Performance Optimization	20
4.4	Design System Diagrams.....	20
4.4.1	DFD Diagram	20
5.	IMPLEMENTATION AND TESTING	21
5.1	Implementation	21
5.1.1	LLM Service Overview	21
5.1.2	Gemini Backend Integration	22
5.1.3	Instruction-to-Action Pipeline.....	23
5.1.4	Visual Context Construction and Validation	24
5.1.5	Action Conversion to HID Commands.....	24
5.1.6	Model Selection and Operational Decision.....	26
5.2	Testing	26
5.2.1	Test Plan and Strategy	26
5.2.2	Test Case Design	27
6.	RESULTS AND DISCUSSIONS.....	37
6.1	Results	37
6.1.1	Overall Execution Outcomes.....	37
6.1.2	Quantitative Performance Summary	40
6.1.3	Accuracy and Output Structure Quality	41
6.1.4	Robustness Under Edge Condition	42
6.1.5	Comparative Behavior Across Scenario Types	43
6.1.6	Practical Result Interpretation	44
6.2	Research Finding	45
6.3	Discussion	46
6.3.1	Interpretation Against Research Objectives	47
6.3.2	Effectiveness of the Proposed Approach	47
6.3.3	Implication of Human-Computer Interaction	47
6.3.4	Reliability-Latency Trade-off	48
6.3.5	Workflow Continuity as a Limitation and Opportunity.....	48
6.3.6	Trust, Safety and Explainability	49
6.3.7	Threats to Validity	49
6.3.8	Practical Deployment Implications	49
7.	CONCLUSIONS.....	50
8.	REFERENCES.....	52
9.	APPENDICES.....	54

LIST OF FIGURES

Figure 4.1: Overview of system architecture diagram	11
Figure 4.2: High-level layered architecture diagram	14
Figure 4.3: LLM component diagram	15
Figure 4.4: LLM processing workflow	17
Figure 4.5: DFD diagram for LLM and Prompt Engineering	20
Figure 5.1: LLM endpoint flow diagram	22
Figure 5.2: Gemini client implementation flow	23
Figure 5.3: LLM prompt call and JSON parsing	24
Figure 5.4: HID commands	25
Figure 6.1: Generated action steps displayed in terminal	38
Figure 6.2: Postman request payload	39

LIST OF TABLES

Table 1.1: Limitations and solution	6
Table 4.1: Technologies, techniques, architecture and algorithms used	13
Table 5.1: Verify login screen action generation	28
Table 5.2: Verify missing UI element handling	28
Table 5.3: Verify ambiguous instruction handling	29
Table 5.4: Verify JSON output correctness	30
Table 5.5: Verify visual-context validation	30

LIST OF ABBREVIATIONS

Abbreviation	Description
LLM	Large Language Model
API	Application Programming Interface
GUI	Graphical User Interface
UI	User Interface
OCR	Optical Character Recognition
HID	Human Interface Device
RPA	Robotic Process Automation
DOM	Document Object Model
DFD	Data Flow Diagram

1. INTRODUCTION

1.1 General Introduction

Screen Aware Task Agent is an agentic artificial intelligence-based automation system designed to interact with GUIs in a human-like manner. The system is developed to support task automation in environments where conventional software integration techniques are limited or infeasible, such as legacy systems, secure platforms, or applications that do not expose APIs. Unlike traditional automation approaches that rely on direct programmatic access, the proposed system operates through external visual observation, interpreting interface states and executing actions via input emulation. This approach aligns with emerging trends in vision-based automation and black-box testing methodologies [1], [2].

The system is based on a closed-loop workflow consisting of perception, planning, action, and evaluation. In the perception stage, an external camera captures the target screen, and computer vision techniques, including optical OCR, are applied to extract textual and structural information from the interface [3]. The planning stage utilizes a LLM to transform user-provided natural language instructions into structured, step-by-step action sequences [4]. These actions are then executed in the action stage through a hardware-based input mechanism using an ESP32 microcontroller configured as a HID, enabling interaction with the target system as if performed by a human user. Finally, the evaluation stage analyzes visual feedback and execution logs to determine task completion and guide subsequent actions if necessary.

To ensure scalability and modularity, the system is implemented using a microservices-based architecture. A FastAPI-based backend functions as the central orchestrator, managing communication between independent services. The vision service is responsible for screen perception and data extraction, the LLM service handles instruction interpretation and action planning, and the HID service manages physical input execution. This architectural approach follows established principles of modular and distributed system design for scalable software engineering [5]. Additionally, a React-based frontend provides a user interface for task input and system monitoring.

The proposed approach demonstrates how the integration of artificial intelligence, computer vision, large language models, and hardware-based input emulation can facilitate flexible and non-intrusive automation. By incorporating context-aware reasoning and visual feedback mechanisms, the system is capable of handling dynamic and unpredictable interface conditions more effectively than traditional rule-based automation methods [6]. Although the system is evaluated within controlled environments, it establishes a foundation for further research in autonomous task execution, adaptive interface interaction, and intelligent human-computer interaction systems.

1.2 Background Literature

Graphical User Interface automation has been studied for decades to improve software quality and reduce repetitive manual interaction. Classical approaches such as WebDriver-based automation and rule-driven RPA pipelines perform effectively when tools have access to internal application structures, including DOM metadata, application programming interfaces (APIs), or accessibility trees [7], [8]. In stable environments, these methods provide high precision and execution speed. However, their robustness decreases significantly in black-box settings where internal hooks are unavailable, unstable, or intentionally restricted [9].

To address the limitations of internal-access dependency, researchers have explored vision-based approaches for interface understanding. Optical Character Recognition techniques are commonly used to extract textual information from screenshots or camera-captured frames, with systems such as Tesseract OCR serving as widely adopted baselines [10]. Furthermore, datasets and studies on interface structure, such as the Rico dataset, have supported data-driven modeling of UI layouts [11]. Despite these advancements, vision-based systems primarily focus on perception and often lack robust high-level reasoning capabilities for multi-step task execution under dynamic interface conditions.

Recent advancements in LLMs, such as Gemini, have significantly expanded the capabilities of intelligent automation systems. These models are capable of interpreting natural language instructions and generating structured action sequences through prompt engineering techniques. Foundational work in few-shot learning and reasoning-based

prompting demonstrates that LLMs can generalize instruction-following behavior and produce intermediate reasoning steps [12], [13]. Agentic frameworks such as React further enhance this capability by integrating reasoning with action execution in iterative loops [14]. However, many LLM-based systems are evaluated in simulated or API-rich environments, which limits their applicability in real-world desktop scenarios involving physical interaction constraints [15].

Hardware-assisted interaction provides an alternative solution for automation in restricted environments. Human Interface Device based techniques utilize microcontrollers, such as ESP32, to emulate keyboard and mouse inputs without requiring software-level integration on the host system [16], [17]. This approach is particularly useful in secure or legacy environments. However, while HID-based execution enables non-intrusive interaction, it typically lacks adaptive intelligence and relies on external logic for decision-making.

Overall, the literature demonstrates significant advancements in the individual domains of perception, reasoning, and execution. Nevertheless, the integration of these components into a unified, closed-loop automation framework for real-world black-box environments remains limited. This limitation highlights the need for an end-to-end system capable of combining these capabilities in a cohesive and adaptive manner.

1.3 Limitation in the Current Approach

Although the proposed system demonstrates the feasibility of screen-aware, LLM-guided automation, several limitations remain in the current implementation.

First, the perception pipeline is sensitive to environmental variability. Factors such as low lighting conditions, screen glare, motion blur, perspective distortion, and camera misalignment can reduce the accuracy of OCR and UI element detection. This limitation arises from the reliance on external camera input rather than direct screen capture, introducing noise and inconsistencies into the perception process. Consequently, errors in visual interpretation may propagate to subsequent planning and execution stages.

Second, the planning module inherits the probabilistic nature of LLM inference. Despite the use of structured prompting and output constraints, ambiguous instructions or

incomplete visual context may result in inconsistent, overly generalized, or partially incorrect action sequences. This variability can affect the reproducibility and reliability of automation outcomes in complex scenarios.

Third, end-to-end latency remains a challenge due to the multi-stage system architecture. The combined processes of image capture, preprocessing, model inference, orchestration, and command transmission introduce delays that can limit responsiveness in time-sensitive applications. Although the adoption of cloud-based LLM services has improved performance, strict real-time operation is not consistently achievable.

Fourth, the execution layer depends on the reliability of hardware-based HID communication. Factors such as USB or Bluetooth instability, variations in host system input handling, and device synchronization issues may lead to delayed or failed action execution. While these challenges are primarily operational, they have a direct impact on overall system performance.

Fifth, the evaluation of the system has been conducted within controlled environments and a limited set of application scenarios. As a result, the generalizability of the system across diverse user interfaces, screen resolutions, languages, and real-world deployment conditions has not been fully validated.

1.4 Research Gap

While Large Language Models (LLMs) demonstrate significant potential for automating GUI interactions through natural language instructions, several limitations restrict their effective application in real-world, black-box testing environments. Although LLMs are capable of understanding user intent and generating action-oriented responses, their outputs are not inherently optimized for direct integration with automated execution systems.

One major limitation is the lack of structured and machine-readable outputs. Many LLMs generate free-form textual responses that are not directly compatible with downstream automation pipelines. In the absence of standardized output formats, such as JSON-based schemas, it becomes challenging to convert high-level instructions into precise, deterministic, and executable UI actions. This limitation introduces ambiguity and

increases the complexity of translating model outputs into reliable system behavior [22], [23], [24].

Another critical challenge is the limited capability for error recovery and adaptive re-planning. In practical automation scenarios, interface states frequently change due to dynamic content, unexpected system responses, or execution failures. However, many existing LLM-based approaches follow a static execution strategy and lack mechanisms for iterative feedback integration. As a result, when an action does not produce the expected outcome, the system is unable to revise its plan dynamically, reducing reliability in real-world environments [19], [25].

Furthermore, current prompt engineering practices are predominantly general-purpose and are not specifically tailored for GUI automation tasks. Effective UI interaction requires precise, context-aware, and stepwise instructions, yet generic prompting strategies often produce inconsistent, incomplete, or non-executable outputs. The absence of domain-specific prompt templates limits the ability of LLMs to generate structured action sequences that align with the requirements of automated UI workflows [21], [22].

In addition, a significant gap exists between high-level reasoning and low-level execution. While LLMs excel at interpreting user intent and generating conceptual plans, there is insufficient research on how prompt engineering can facilitate the transformation of these high-level instructions into precise, low-level interaction commands, such as mouse movements and keyboard inputs. This disconnect between reasoning and execution presents a major barrier to deploying LLM-based systems in real-world automation contexts [18], [20].

Moreover, existing approaches often evaluate LLM capabilities in controlled or simulated environments, where assumptions such as clean input data, stable interface layouts, and predictable execution conditions are maintained. In contrast, real-world black-box environments introduce additional challenges, including noisy visual inputs, partial observability, and limited system feedback, which are not adequately addressed in current research.

Therefore, a clear research gap exists in the development of a prompt-engineering-driven LLM framework that can generate structured, machine-readable outputs, support adaptive re-planning through iterative prompting, and effectively bridge the gap between high-level intent and executable UI actions in dynamic, real-world environments.

Table 1. 1: Limitations and solutions

Aspect	Current Limitation	Impact	Proposed Direction
Output Format	Unstructured free-text outputs	Difficult to parse and execute	Use structured formats (e.g., JSON templates)
Error Handling	Limited re-planning after failures	Fragile execution in dynamic UIs	Apply adaptive re-prompting with feedback
Prompt Design	General-purpose prompts	Inconsistent or non-executable steps	Use domain-specific prompt templates
Reasoning–Execution Mapping	Weak mapping from intent to actions	Plans not directly executable	Add deterministic action-mapping layer

The proposed Screen-Aware Task Agent addresses this gap by introducing an LLM-driven reasoning and prompt engineering framework that produces structured action plans, incorporates feedback-driven re-planning, and enables the transformation of high-level user instructions into precise and executable UI interaction commands. This approach is specifically designed to operate within real-world, access-restricted environments, thereby improving the reliability, adaptability, and practical applicability of LLM-based automation systems.

2. RESEARCH PROBLEM

2.1 Problem Context

Large Language Models (LLMs) have recently emerged as powerful tools for interpreting natural language instructions and generating task-oriented action sequences. Their ability to understand user intent, process contextual information, and produce structured responses makes them highly suitable for automation scenarios where tasks are defined using human-readable instructions. This capability enables a shift from traditional rule-based automation toward more flexible and intelligent systems that can adapt to varying user requirements.

However, applying LLMs to real-world automation tasks, particularly in GUI environments, introduces several challenges. Unlike simple text-based tasks, GUI automation requires precise, context-aware, and sequential decision-making, where each action must be correctly aligned with the current interface state. These environments are often dynamic, containing changing layouts, varying visual elements, and unpredictable system responses. As a result, automation systems must go beyond static instruction generation and support continuous reasoning, contextual understanding, and adaptive behavior throughout task execution.

2.2 Problem Statement

Despite their advanced capabilities, existing LLM-based automation approaches exhibit several critical limitations when applied to GUI-based environments. One major issue is that LLMs frequently generate unstructured or semi-structured outputs, which are not directly compatible with automated execution systems. This creates a gap between high-level reasoning and practical implementation, requiring additional processing to convert outputs into executable actions.

In addition, LLMs often demonstrate weak contextual persistence across multi-step interactions. In complex workflows, where tasks require maintaining state and referencing previous actions, the model may lose track of context, leading to inconsistencies or incorrect action sequences. This significantly affects the reliability of automation in real-world scenarios.

Furthermore, most existing implementations rely on general-purpose prompting strategies that are not specifically designed for GUI automation tasks. These approaches often fail to enforce structured outputs or provide sufficient constraints, resulting in ambiguous, incomplete, or non-actionable responses. The absence of domain-specific prompt engineering reduces the effectiveness of LLMs in generating precise and deterministic UI interaction steps.

Another important limitation is the lack of adaptive re-planning capabilities. In dynamic environments, execution outcomes may deviate from expectations due to interface changes, delays, or detection errors. However, many current systems lack mechanisms to incorporate feedback and revise their plans accordingly. This limits their ability to operate reliably in real-world conditions where uncertainty and variability are common.

2.3 Core Research Problem

Therefore, the core research problem addressed in this study is the absence of a robust and integrated LLM-driven reasoning and prompt engineering framework that can effectively bridge the gap between high-level instruction understanding and low-level executable actions. Such a framework must be capable of generating structured, machine-readable action sequences, maintaining contextual consistency across multi-step interactions, and dynamically adapting to changing conditions through feedback-driven re-planning.

Addressing this problem is essential for enabling reliable and scalable automation in complex environments, where systems must operate with minimal assumptions, handle ambiguity in user instructions, and respond intelligently to evolving interface states. The development of such a framework represents a critical step toward achieving practical, human-like automation in real-world applications.

3. RESEARCH OBJECTIVES

3.1 Main Objective

The primary objective of this study is to develop a robust LLM-driven reasoning engine supported by advanced prompt engineering techniques, capable of transforming high-level natural language testing instructions into structured and executable UI action sequences. The system is designed to bridge the gap between human intent and machine-executable commands by enabling accurate interpretation, structured planning, and reliable execution of UI-based tasks.

Furthermore, the proposed solution aims to support adaptive, context-aware, and human-like automation across both functional and exploratory testing scenarios. In functional testing, the system must follow predefined instructions with high precision, while in exploratory testing, it must demonstrate flexibility in navigating unknown or dynamic interface states. By operating in black-box environments where internal application access is unavailable, the system ensures platform independence and broad applicability across diverse software systems. Ultimately, the objective is to create an intelligent automation framework that can continuously interpret, plan, and respond to evolving interface conditions in a reliable and scalable manner.

3.2 Specific Objective

The specific objectives of this study are to design domain-specific, context-aware prompt templates that enhance the LLM's ability to accurately interpret diverse and potentially ambiguous user instructions while ensuring consistent and structured output generation. These prompt templates are intended to guide the model toward producing precise, machine-readable action sequences that can be directly utilized by downstream execution components.

In addition, the study aims to implement instruction parsing and task decomposition mechanisms that convert high-level user goals into ordered and granular UI action steps.

This decomposition enables complex tasks to be broken down into manageable sub-actions, improving execution reliability and facilitating systematic automation workflows.

Furthermore, the research focuses on integrating LLM-generated action plans with real-time visual perception data. By grounding decisions in the current state of detected interface elements, the system can ensure that generated actions are contextually relevant and aligned with the actual UI environment, thereby reducing execution errors.

Another key objective is to develop a feedback-driven re-prompting mechanism that continuously evaluates execution outcomes and supports adaptive plan refinement. This mechanism enables the system to detect failures, interpret discrepancies, and dynamically update its action strategy, thereby improving robustness and enabling autonomous error recovery in changing environments.

Additionally, the study aims to optimize prompt engineering strategies to effectively support both deterministic functional testing and flexible exploratory testing. This involves balancing precision and adaptability, allowing the system to execute predefined tasks accurately while also exploring and interacting intelligently with unfamiliar interface states.

Finally, the system is designed to ensure platform-independent, black-box operation by relying solely on external observations and user-provided instructions. By eliminating dependence on internal APIs, DOM structures, or application-level hooks, the proposed approach enhances compatibility with legacy systems, secure platforms, and cross-environment applications.

4. METHODOLOGY

The methodology for the LLM and prompt engineering module in the Human-Like Agent for Screen-Aware Task Automation project is designed to transform natural language test instructions into actionable, screen-aware UI interaction plans. The process involves multiple integrated steps: input processing, prompt engineering, action generation, execution feedback, and dynamic adaptation.

4.1 System Architecture

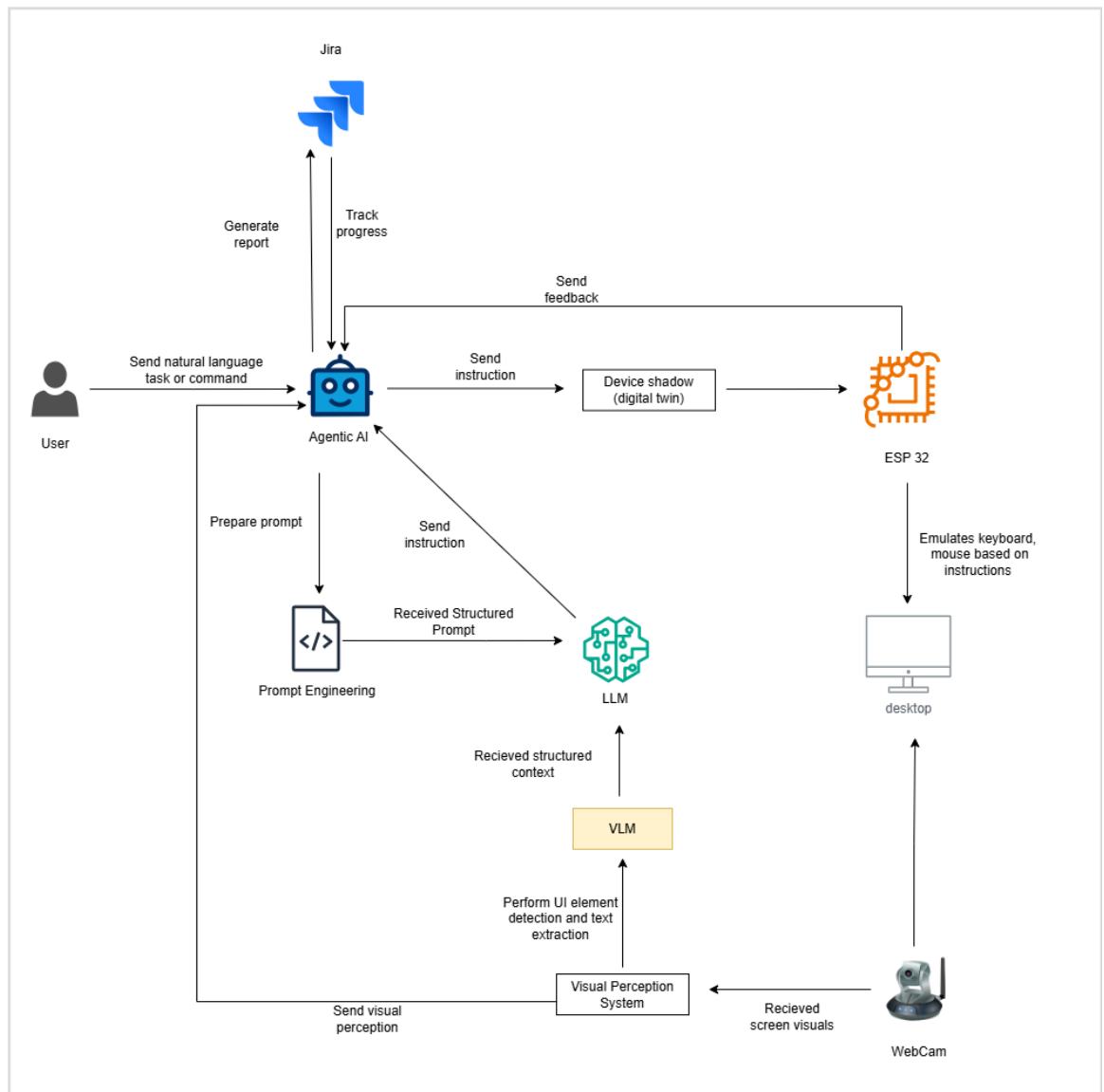


Figure 4. 1: Overview of system architecture diagram

Figure 4.1 presents the end-to-end architecture of the Screen-Aware Task Agent as a closed-loop, agentic automation system designed for black-box GUI testing. The architecture is intentionally built around external observation and hardware-level interaction so that testing can be performed even when internal application APIs, source code, or DOM hooks are unavailable. At a high level, the system follows a continuous perceive-plan-act-evaluate cycle, where each execution step is validated against the latest visual state before the next action is issued.

The workflow starts at the user interaction layer, where a tester provides a high-level natural-language objective (for example, changing a setting or validating a workflow). This instruction is received by the Agentic AI controller, which functions as the central orchestration unit. Rather than executing actions directly from raw text, the controller delegates interpretation to the prompt engineering and LLM pipeline. In this stage, the original user instruction is enriched with execution context, such as current UI conditions, historical actions, constraints, and expected output format. The LLM then transforms this context-aware prompt into structured, machine-readable action steps that can be executed deterministically.

In parallel with language reasoning, the perception path continuously captures the target desktop using an external webcam. The visual perception module processes each frame through UI element detection and OCR to identify interactable controls and visible textual cues. The VLM layer further semantically interprets this data, converting raw detections into structured scene context that is meaningful for planning (for example, identifying likely buttons, menu states, or confirmation messages). This perception output is fed back to both the LLM and the Agentic AI controller, ensuring that planning is grounded in what is actually visible on screen rather than assumed interface states.

After a valid step plan is produced, the Agentic AI controller sends execution commands to the device-shadow layer, which acts as an interface bridge to physical actuation. The device shadow forwards low-level commands to the ESP32 HID

module, which emulates standard keyboard and mouse input on the target machine. This hardware-in-the-loop design is a key architectural strength: it enables non-intrusive, platform-agnostic interaction that mirrors human operation and preserves black-box testing realism.

Once an action is executed, the system does not assume success. The webcam captures the updated screen, and the perception stack re-evaluates the new state. The controller compares expected versus observed outcomes and decides whether to continue, retry, re-plan, or branch to an alternative path. This verification-driven control loop supports robust behavior under dynamic UI conditions, layout changes, delayed responses, or unexpected dialogs. As a result, the system can adapt online instead of failing on first mismatch.

The architecture also includes an observability and project-management integration layer. Execution traces, action logs, state transitions, and completion outcomes can be synchronized with Jira to support test traceability, progress monitoring, and report generation. This improves accountability and makes the system practical for real testing pipelines, where reproducibility and auditability are as important as execution success.

Summary of technologies, techniques, architecture and algorithm used for effective resource is shown in the table 4.1 below.

Table 4. 1: Technologies, techniques, architecture and algorithms used

Technologies	FastAPI + React, Gemini, OCR (Tesseract), ESP32 HID, JSON
Techniques	Prompt engineering, context injection, instruction normalization, feedback-driven re-prompting
Algorithms	Task decomposition, UI state comparison, rule-based output validation, retry and recovery logic
Architectures	Microservices architecture, agentic closed-loop

4.2 illustrates the high-level layered architecture system diagram of the proposed system.

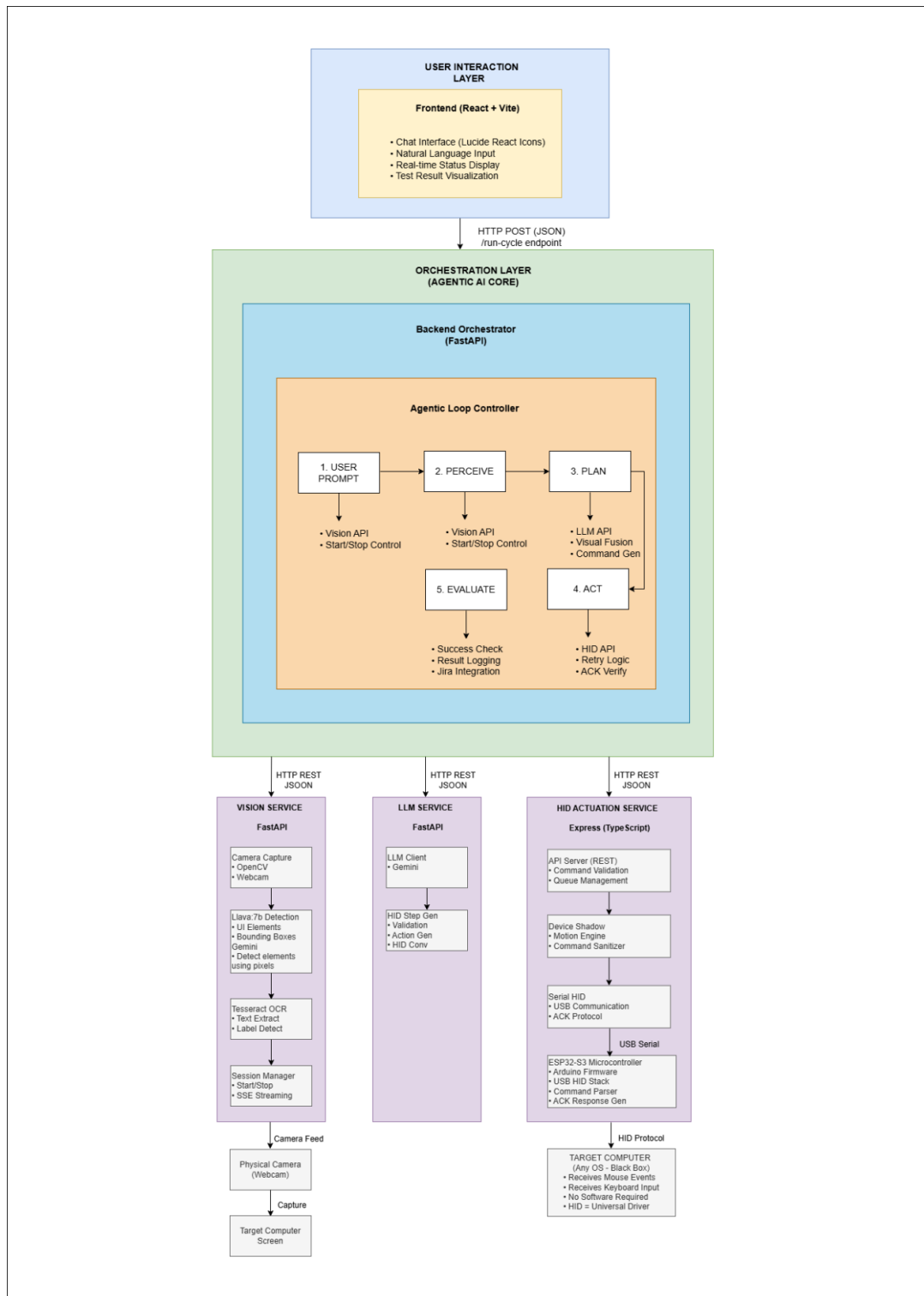


Figure 4. 2: High-level layered architecture diagram

4.2 Specific Component Description

The LLM and Prompt Engineering module is the central intelligence unit responsible for transforming high-level natural language instructions into structured, executable UI action plans for the Agentic AI Core. This component acts as the bridge between human intent and machine execution, ensuring that the system can understand, plan, and adapt in a human-like manner during testing.

4.2.1 Purpose

The LLM and Prompt Engineering module interprets natural language test commands and converts human intent into actionable system instructions. It enriches each request with relevant context, such as current UI state, prior actions, and testing goals, to keep decisions grounded and accurate. The module then produces step-by-step action plans in machine-readable formats like JSON for reliable execution. It also adapts dynamically by detecting unexpected UI changes and revising plans in real time to maintain robust performance.

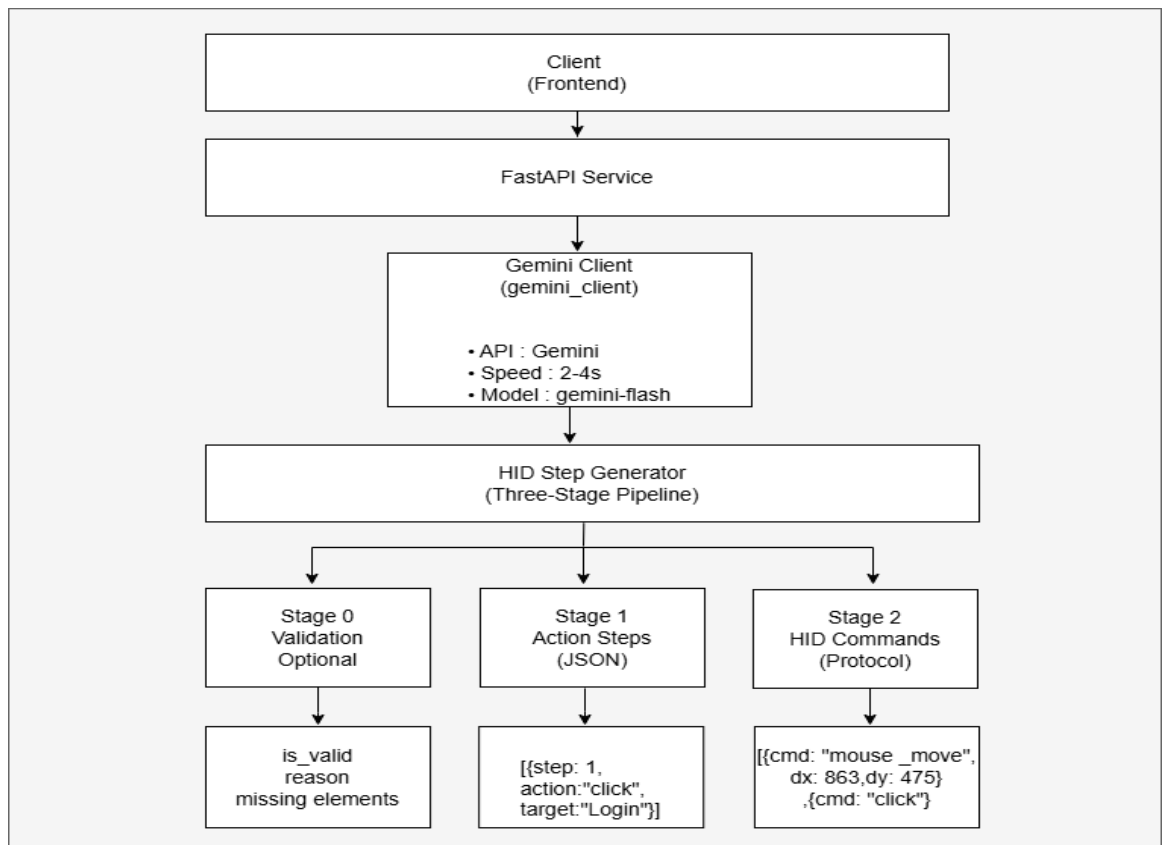


Figure 4. 3: LLM component diagram

4.2.2 Workflow Overview

The module operates through a comprehensive pipeline that processes user instructions into executable actions:

Instruction Intake & Pre-processing: The system accepts high-level test instructions from the user (e.g., "Open the settings menu and reduce the volume to 50%"). These inputs are normalized to remove ambiguities and ensure consistent interpretation by the LLM.

Context-Aware Prompt Construction: Using pre-designed prompt templates, the system injects relevant task context, constraints, and desired output formatting. This contextual information includes the current UI state from the visual perception module, operational constraints (e.g., time limits, non-destructive operations), and specifications for the desired output structure. Prompts are iteratively refined to balance specificity and flexibility.

LLM Processing & Action Plan Generation: The prepared prompt is sent to an LLM, which decomposes complex tasks into smaller atomic actions and outputs them in structured, machine-readable format (JSON). Each action is specifically grounded to on-screen UI elements detected by the vision module, ensuring precise interaction with correct components.

Dynamic Adaptation & Feedback Loop: The system monitors real-time feedback from the visual perception module after each action. If discrepancies or unexpected elements are detected, the system re-invokes the LLM with updated context to adjust the action plan dynamically. Post-action screen captures and state analysis determine if actions succeeded or if deviations occurred.

Exploratory Testing Support: When operating in exploratory mode, the module can suggest alternative interaction paths or test cases based on observed UI changes, simulating human tester curiosity and creativity. Fail-safe prompt templates handle ambiguous or failed states by prompting the LLM to explore alternative paths or request clarifications.



Figure 4. 4: LLM processing workflow

4.2.3 Input Processing

Natural language commands are accepted at the system interface, such as "Open the settings menu and change the screen brightness to 70%." The pre-processing stage normalizes these inputs, removing ambiguities and standardizing terminology to facilitate consistent interpretation by the LLM.

4.2.4 Prompt Engineering

Prompt templates form the foundation of reliable LLM interaction. These templates embed task context, UI state information, constraints, and desired output formatting into the input command before querying the LLM. This contextual embedding ensures that the LLM has sufficient information to generate accurate, contextually appropriate action plans. Templates are continuously refined based on execution feedback to optimize the balance between specificity and adaptability.

4.2.5 Instruction Parsing and Action Plan Generation

The prepared prompt is submitted to the LLM, which generates a step-by-step UI action plan in machine-readable format (JSON or XML). Complex user commands are automatically decomposed into smaller atomic steps, with each step detailing specific UI interactions (e.g., "click button X," "enter text field Y"). This decomposition facilitates precise execution and enables the system to handle failures at a granular level.

4.2.6 Integration with Visual Perception Module

Generated actions are grounded to actual on-screen UI elements detected by the vision module, ensuring the agent interacts with correct components. The visual perception

module verifies UI state before and after each action to confirm expected outcomes. This bidirectional integration creates a closed-loop system where perception informs planning and execution feedback informs future perceptual analysis.

4.2.7 Execution and Error Handling

The Agentic AI Core executes LLM-generated UI actions through appropriate input emulation (mouse, keyboard, touch). Post-execution analysis compares observed UI state against expected outcomes. If failures or unexpected states are detected, the system re-prompts the LLM with updated context and feedback, enabling dynamic recovery from execution errors. Special prompt templates guide the LLM through ambiguous or failed states, promoting adaptive exploration of alternative interaction paths.

4.2.8 Exploratory Branching

For exploratory testing scenarios, the LLM is encouraged to generate creative action sequences based on observed UI changes. This capability enables the system to discover edge cases and unexpected behaviors, mimicking the intuitive branching behavior of experienced human testers. The module supports generating test cases that explore alternative UI paths and interaction sequences.

4.2.9 Optimization and Evaluation

Continuous improvement of prompt templates is driven by execution feedback and test outcomes. Performance is evaluated across multiple dimensions: accuracy of generated action plans, success rate of task completion, adaptability to UI state changes, and reduction in human intervention requirements. These metrics collectively assess the module's effectiveness in bridging natural language intent and precise machine execution.

4.3 Prompt Engineering and LLM Action Planning Framework

This section presents the methodology used for the LLM and prompt engineering module in the proposed system. The objective is to transform natural-language test instructions into reliable, structured UI action plans while adapting to dynamic interface changes. Although

the system architecture supports multiple LLM backends, this implementation primarily uses Gemini due to lower response latency and more stable step generation performance in iterative execution.

4.3.1 LLM Input Formulation

User instructions are normalized to reduce ambiguity and improve consistency. The cleaned instruction is combined with contextual signals, including current UI state, prior actions, and execution constraints, to produce a model-ready request.

4.3.2 Prompt Template Strategy

Predefined prompt templates are used to standardize model reasoning behavior. Each template defines instruction framing, context injection format, output schema requirements, and safety constraints, ensuring predictable and machine-readable responses.

4.3.3 Reasoning and Task Decomposition

The LLM interprets the contextual prompt and decomposes high-level objectives into atomic UI actions. Each action includes explicit intent, target reference, and expected outcome to support deterministic execution.

4.3.4 Structured and Action Plan

Generated plans are formatted as JSON-based step sequences. Each step contains ordered actions, parameters, and optional verification checks, enabling direct handoff to the execution layer.

4.3.5 Output Validation and Correction

Before execution, action plans are validated for schema correctness, safety compliance, and executability. If a plan is invalid or incomplete, the system triggers automatic re-prompting with corrective context.

4.3.6 Feedback Aware Re-prompting

After each executed step, observed UI feedback is compared with expected outcomes. On mismatch, updated state information is injected and the LLM produces a revised plan for adaptive recovery.

4.3.7 LLM Performance Optimization

Prompt templates are iteratively refined using execution logs and failure patterns. Optimization focuses on reducing ambiguity, minimizing re-prompts, improving plan accuracy, and increasing robustness under real UI variability.

4.4 Design System Diagrams

System design diagrams were developed to identify the essential components and organize the implementation of the proposed model. Figure 4.4 illustrates DFD diagram to helped to develop this research component.

4.4.1 DFD Diagram

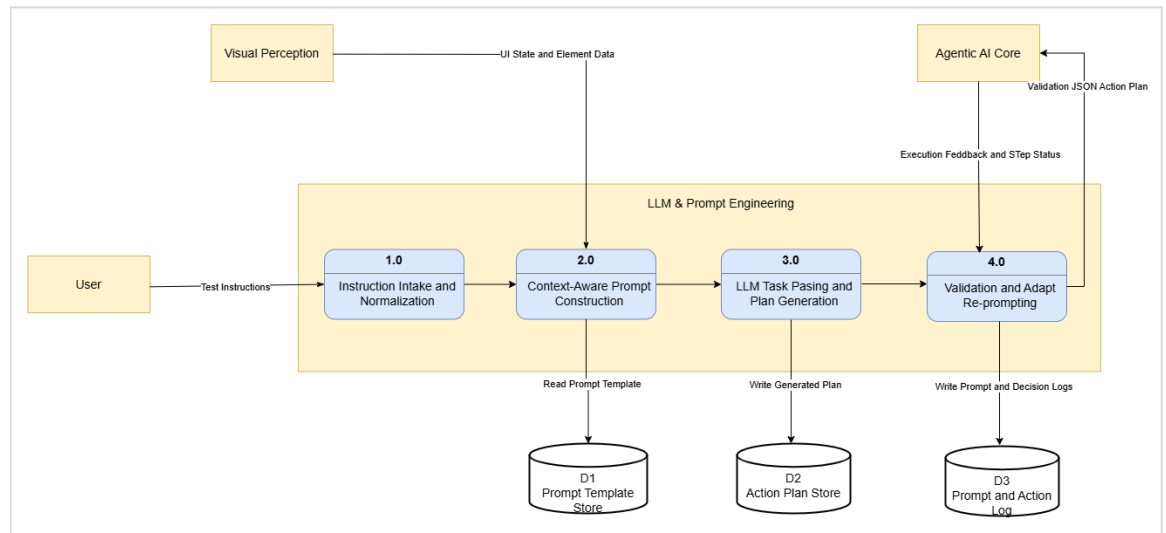


Figure 4. 5: DFD diagram for LLM and Prompt Engineering

5. IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation focuses on a screen-aware, LLM-driven automation pipeline that transforms natural-language user instructions into structured UI actions. The LLM module is implemented as a FastAPI microservice that accepts task instructions, applies prompt-based reasoning, and returns machine-readable plans. In the final operational setup, Gemini is used as the primary and active language model backend due to lower response latency and more stable iterative step generation in closed-loop execution. The implementation includes visual-context grounding, where detected screen information is converted into textual context and combined with user intent before generation. Generated steps are then validated, refined, and converted into HID commands for execution by the Agentic AI system on the target machine. This implementation supports a continuous perceive-plan-act-evaluate loop for real-world black-box GUI automation.

5.1.1 LLM Service Overview

The LLM microservice serves as the main reasoning component of the system and is responsible for converting user instructions into structured, executable actions. It is implemented as a FastAPI service that exposes endpoints for generation, validation, health checking, and runtime status monitoring. When an instruction is received, the service first validates the request, then builds or routes it through the appropriate generation pipeline, produces structured action steps, and converts them into HID-compatible commands for execution on the target machine. In the final implementation, Gemini is used as the primary backend because it provides lower response latency and more stable iterative generation, while the service still retains support for alternative backends in the architecture. This microservice-based design allows the LLM component to operate independently while remaining tightly integrated with the visual perception and HID execution layers.

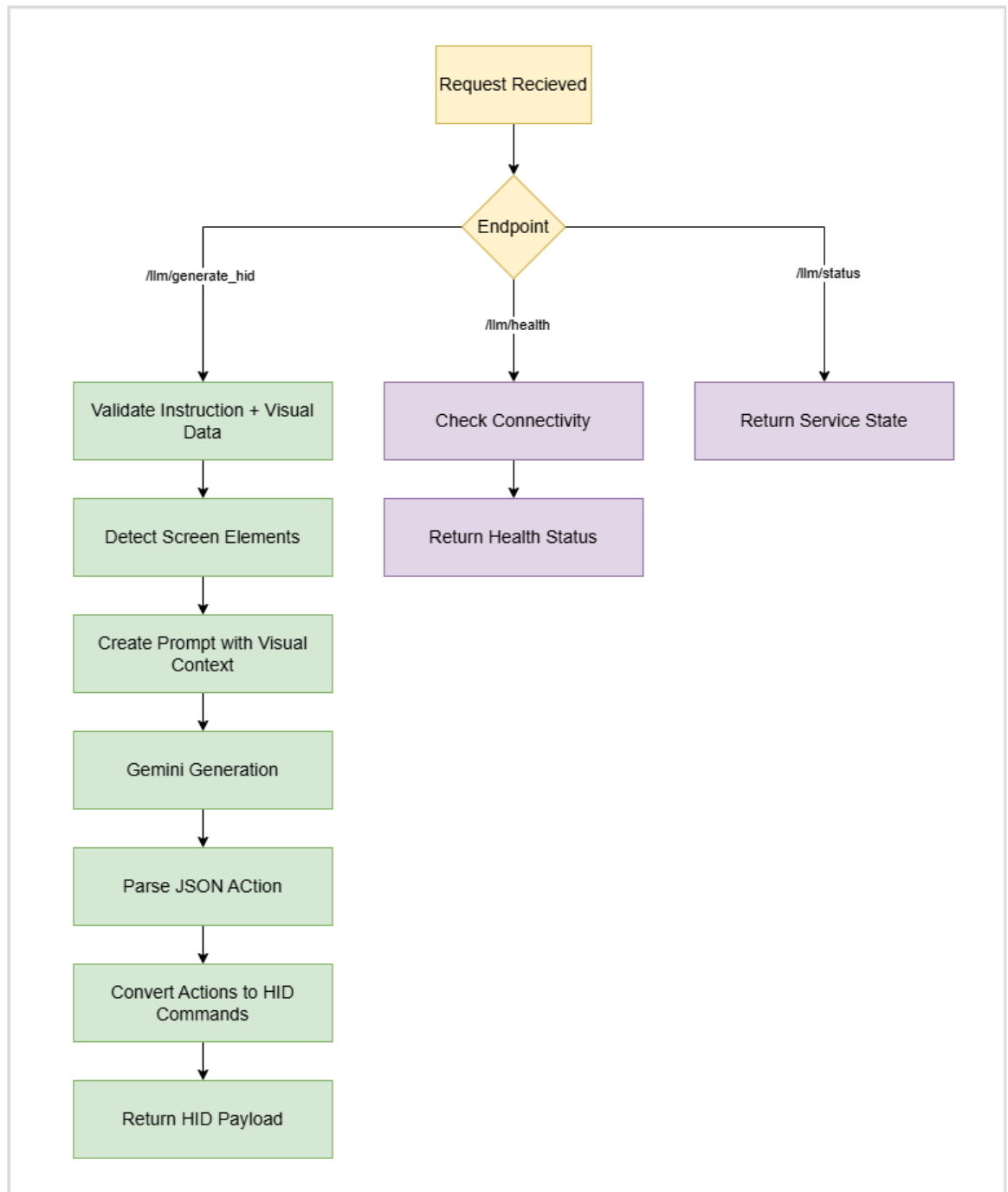


Figure 5.1: LLM endpoint flow diagram

5.1.2 Gemini Backend Integration

Gemini is integrated through a dedicated client wrapper that encapsulates API-key handling, model initialization, generation configuration, and response retrieval. This abstraction allows the LLM service to call Gemini through a stable internal interface

while keeping model-specific details isolated from endpoint logic. In the final operational pipeline, Gemini is used as the primary backend due to its lower response latency and more stable iterative generation behavior in closed-loop execution. The wrapper-based integration also improves maintainability by allowing model parameters and provider logic to be updated without changing core orchestration code.

```
llm > gemini_client.py > ...
23 class GeminiClient:
24     """Google Gemini API client matching OllamaClient interface"""
25
26     def __init__(self, api_key: Optional[str] = None):
27         """
28         Initialize Gemini client.
29
30         Args:
31             api_key: Google API key. If None, reads from GEMINI_API_KEY or GOOGLE_API_KEY env var
32         """
33         self.api_key = api_key or os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")
34         if not self.api_key:
35             logger.warning("⚠ No Gemini API key provided. Set GEMINI_API_KEY env var or pass api_key parameter.")
36
37         # Lazy import to avoid dependency if not used
38         self._genai = None
39
40     def _ensure_genai(self):
41         """Lazy import of google.generativeai library"""
42         if self._genai is None:
43             try:
44                 import google.generativeai as genai
45                 self._genai = genai
46                 if self.api_key:
47                     self._genai.configure(api_key=self.api_key)
48             except ImportError:
49                 logger.error("❌ google-generativeai library not installed. Run: pip install google-generativeai")
50                 raise ImportError("google-generativeai library required for GeminiClient. Install with: pip install google-generativeai")
51
52     def generate(
53         self,
```

Figure 5.2: Gemini client implementation flow

5.1.3 Instruction-to-Action Pipeline

The instruction-to-action pipeline converts high-level natural-language tasks into structured, executable action plans. The process starts by constructing a constrained prompt that defines output schema and interaction rules, then invokes the LLM to generate JSON-formatted action steps. The response is parsed, sanitized, and post-processed to ensure usability, including correction strategies for incomplete or placeholder fields. This pipeline is a key implementation component because it transforms ambiguous human intent into deterministic action objects that can be validated and executed programmatically. It also provides traceable intermediate artifacts such as rewritten steps and structured action arrays.

```

603 Output JSON array:'''
604
605     logger.info(f"Stage 1: Generating action plan for: {instruction[:50]}...")
606
607     llm_output = self.client.generate(
608         prompt=prompt,
609         model=model,
610         max_tokens=max_tokens,
611         timeout=30
612     )
613
614     logger.info(f"Stage 1 LLM output:\n{llm_output}")
615
616     # Parse JSON output
617     try:
618         # Strip markdown code blocks if present
619         import re
620         clean_output = llm_output.strip()

```

Figure 5.3: LLM prompt call and JSON parsing

5.1.4 Visual Context Construction and Validation

Before generating actions, the system transforms visual perception outputs into compact, context-aware textual summaries that describe on-screen interactive elements and coordinates. This contextual representation is injected into LLM prompts to ground planning in the actual current UI state. A validation stage then checks whether the instruction is feasible with available elements and reports confidence, reasoning, and missing targets when applicable. This mechanism reduces invalid plans and improves robustness under dynamic interface conditions by ensuring that generation is aligned with what is truly visible at runtime.

5.1.5 Action Conversion to HID Commands

After structured action steps are produced, they are translated into low-level HID operations suitable for physical execution on the target machine. High-level actions such as click, type, and key press are mapped into protocol-compliant command sequences including `mouse_move`, `mouse_click`, `type_text`, and key events. This conversion layer bridges semantic planning and hardware-level interaction, enabling the Agentic AI core to execute tasks through ESP32-based input emulation. The mapping strategy ensures reproducibility and preserves a clear relationship between generated intent and final executable commands.

Figure 5.4 illustrates the process of converting actions into HID commands.

```

73  "hid_commands": [
74      {
75          "cmd": "mouse_move",
76          "meta": {
77              "commandId": "195be7aa-26c9-42a8-9266-89778910bee1"
78          },
79          "dx": 863,
80          "dy": 475,
81          "smooth": true
82      },
83      {
84          "cmd": "mouse_click",
85          "meta": {
86              "commandId": "c53d4ea3-67df-46cc-a42b-e69a1b99c5e0"
87          },
88          "button": "left"
89      },
90      {
91          "cmd": "type_text",
92          "meta": {
93              "commandId": "339a416b-7026-4ed5-802b-4acfa83b776a"
94          },
95          "text": "tharushi"
96      },
97      {
98          "cmd": "mouse_move",
99          "meta": {
100              "commandId": "55d2ad1d-4242-4995-88e5-dd06710aecbf"
101          },
102          "dx": 863,
103          "dy": 583,
104          "smooth": true
105      },
106      {
107          "cmd": "mouse_click",
108          "meta": {
109              "commandId": "6db41eb8-beac-4553-ad41-72f45126d1d6"
110          },
111          "button": "left"
112      },
113      {
114          "cmd": "type_text",
115          "meta": {
116              "commandId": "cb80096c-2e32-4248-8a30-5288e227690e"
117          },
118          "text": "123"
119      },
120      {
121          "cmd": "mouse_move",
122          "meta": {
123              "commandId": "dfd283cf-0f89-419a-b2fc-960cc2d62d82"
124          },
125          "dx": 912,
126          "dy": 739,
127          "smooth": true
128      },
129      {
130          "cmd": "mouse_click",
131          "meta": {
132              "commandId": "2a9215e6-33ac-49ce-816a-f9146c7f377a"
133          },
134          "button": "left"
135      }
136  ],

```

Figure 5.4: HID commands

5.1.6 Model Selection and Operational Decision

Although the architecture is designed to be backend-flexible, the final runtime path is standardized on Gemini for production execution. This decision was based on operational behavior observed in iterative closed-loop use, where response latency and generation stability directly affect end-to-end automation performance. Standardizing the active backend reduces variability in execution flow and improves predictability of generated plans and HID outputs. The model selection strategy is therefore not only architectural but also practical, aligning backend choice with real-time performance requirements of screen-aware automation.

5.2 Testing

Testing was conducted to verify the correctness, robustness, and operational readiness of the LLM-driven screen-aware automation pipeline. The testing scope covered API reliability, prompt-to-action generation quality, validation behavior, HID command conversion, and runtime performance under iterative execution. Because the final operational backend is Gemini, all primary test scenarios were executed using Gemini-based generation paths to ensure consistency with deployment behavior.

The testing process evaluated both component-level behavior (input validation, structured output generation, command mapping) and end-to-end behavior (instruction + visual context -> validated action plan -> HID command sequence). In addition to functional verification, non-functional aspects such as response stability and execution latency were also checked to ensure suitability for closed-loop automation tasks.

5.2.1 Test Plan and Strategy

The A structured and systematic testing strategy was adopted to ensure reliable and repeatable evaluation of the Gemini-based LLM subsystem. The testing process began with the identification of key test items, including LLM request handling, visual-context validation, action-step generation, Human Interface Device (HID) command mapping, and runtime status endpoints. These components were selected because they

directly influence the correctness and effectiveness of the screen-aware automation pipeline.

Priority was assigned to critical functional paths, particularly the endpoint, the accuracy of JSON-based action outputs, and the integrity of generated HID commands, as these elements directly affect real-world execution. Scenario-based testing was employed to evaluate system robustness under diverse conditions. Test scenarios included valid workflows such as login task execution, as well as edge cases involving missing UI elements, ambiguous or malformed instructions, and coordinate-sensitive interactions.

All test cases were executed in controlled environments using API-based requests with representative visual input data. During execution, multiple performance and correctness metrics were recorded, including HTTP response status, output schema validity, validation confidence scores, number of generated action steps, HID command count, and overall execution time.

In addition, fault tolerance was assessed by introducing invalid inputs, incomplete visual data, and non-compliant or incomplete model responses to observe system behavior under failure conditions. Based on the observed results, iterative refinements were applied to prompt design, output validation mechanisms, and error-handling strategies in order to improve consistency, reliability, and execution accuracy of the Gemini-driven LLM subsystem.

The strategy confirmed that the Gemini-based LLM service can consistently generate valid, executable outputs for screen-aware automation tasks.

5.2.2 Test Case Design

The following test cases were designed to ensure system reliability by testing all system functionalities.

Table 5. 1: Verify login screen action generation

Test Case Id	01
Test Case	Verify login screen action generation
Test Scenario	Verify that the LLM generates valid structured steps and HID commands for a login task using visual context.
Input	Natural-language login instruction + visual data with Username, Password, and Login button
Expected Output	1. Validation should pass 2. action_steps, and hid_commands should be generated correctly
Actual Result	1. Validation passed 2. 5 action steps were generated, and 8 HID commands were produced
Status (Pass/Fail)	Pass

Table 5. 2: Verify missing UI element handling

Test Case Id	02
Test Case	Verify missing UI element handling
Test Scenario	Verify that the LLM detects missing required elements and returns constrained output when the Login button is not visible.
Input	Login instruction with visual data containing Username and Password fields only (Login button missing)
Expected Output	1. Validation should fail (is_valid=false) 2. Missing element should be reported, suggested action should be returned, and generated actions should be limited to available UI elements

Actual Result	1. Validation failed as expected (is_valid=false, confidence=1.0) 2. Missing element identified as “Login button,” recovery suggestion returned. Output was constrained to available element interaction (1 action step, 2 HID commands).
Status (Pass/Fail)	Pass

Table 5. 3: Verify ambiguous instruction handling

Test Case Id	03
Test Case	Verify ambiguous instruction handling
Test Scenario	Verify that the LLM can interpret a vague instruction and still generate structured executable actions.
Input	Instruction: “Do the login process properly.” with visual data containing Username, Password, and Login elements
Expected Output	1. Validation should pass 2. Structured rewritten_steps, action_steps, and hid_commands should be generated
Actual Result	1. Validation passed (is_valid=true, confidence=1.0) 2. 5 structured action steps and 8 HID commands were generated successfully
Status (Pass/Fail)	Pass

Table 5. 4: Verify JSON output correctness

Test Case Id	04
Test Case	Verify JSON output correctness
Test Scenario	Verify that the LLM returns valid, machine-readable structured JSON for execution.
Input	Instruction: “Search for ‘wireless mouse’ using the search bar and open the first result.” with visual data containing Search input, Search button.
Expected Output	1. Status should be success 2. Validation.is_valid should be true and structured rewritten_steps 3. Action_steps should be generated in valid JSON format and hid_commands should be generated in executable sequence
Actual Result	1. Status was success 2. Validation.is_valid was true (confidence = 1.0) 3. 4 rewritten_steps and 4 action_steps were generated in valid JSON format, 7 hid_commands were generated successfully (mouse_move, mouse_click, type_text) and execution time recorded as 129.12s
Status (Pass/Fail)	Pass

Table 5. 5: Verify visual-context validation

Test Case Id	05
Test Case	Verify visual-context validation

Test Scenario	Verify that the LLM validates the instruction against the visible UI elements before generating actions.
Input	Login instruction with Username, Password, and Login button in visual data. Request Payload <pre>{ "instruction": "Enter username 'nimal' in the Username field, enter password '123' in the Password field, then click the Login button.", "visual_data": { "session_data": { "screens": [{ "screen_index": 0, "timestamp": "2026-03- 02T10:30:00.000000", "elements": [{ "type": "input", "label": "Username", "dx": 863, "dy": 475, "confidence": 0.95, "source": "layout_form", "state": "enabled" }, { "type": "input", "label": "Password",</pre>

	<pre> "dx": 863, "dy": 583, "confidence": 0.94, "source": "layout_form", "state": "enabled" }, { "type": "button", "label": "Login", "dx": 912, "dy": 739, "confidence": 0.96, "source": "layout_form", "state": "enabled" }] }] } }, "use_gemini": true, "gemini_model": "models/gemini-flash-latest", "skip_validation": false } </pre>
Expected Output	1. Validation should pass with high confidence 2. Structured steps plus HID commands should be generated

Actual Result	1. Validation passed (is_valid=true, confidence=1.0); all required elements were detected 2. 5 rewritten steps, 5 action steps, and 8 HID commands were generated; execution time was 7.12s <pre> { "status": "success", "instruction": "Enter username 'nimal' in the Username field, enter password '123' in the Password field, then click the Login button.", "validation": { "is_valid": true, "confidence": 1.0, "reason": "All required elements (Username input field, Password input field, and Login button) are present on the screen to complete the instruction.", "missing_elements": [], "suggested_actions": [] }, "rewritten_steps": [{ "step": 1, "action": "click", "description": "Click Username input field" }, { "step": 2, "action": "type_text", "description": "Type 'nimal' in Username input field" }, { "step": 3, "action": "click", "description": "Click Password input field" }, { "step": 4, "action": "type_text", "description": "Type '123' in Password input field" }] } </pre>
---------------	--

	<pre> }, { "step": 5, "action": "click", "description": "Click Login button" }], "action_steps": [{ "step": 1, "action": "click", "target": "Username input field", "x": 863, "y": 475 }, { "step": 2, "action": "type_text", "target": "Username input field", "text": "nimal" }, { "step": 3, "action": "click", "target": "Password input field", "x": 863, "y": 583 }, { "step": 4, "action": "type_text", "target": "Password input field", "text": "123" }, { "step": 5, "action": "click", "target": "Login button", "x": 912, "y": 739 }], "hid_commands": [{ </pre>
--	--

	<pre> "cmd": "mouse_move", "meta": { "commandId": "69ab9bb1-f144-4186-94db-178f40cc61cf" }, "dx": 863, "dy": 475, "smooth": true }, { "cmd": "mouse_click", "meta": { "commandId": "6542a994-9a16-4bd4-a09d-1a31313a5854" }, "button": "left" }, { "cmd": "type_text", "meta": { "commandId": "f638604e-38d3-4c7f-8139-1b9a85b35f0c" }, "text": "tharushi" }, { "cmd": "mouse_move", "meta": { "commandId": "81535cc2-30ff-4160-9b71-8e877a64c6d4" }, "dx": 863, "dy": 583, "smooth": true }, { "cmd": "mouse_click", "meta": { "commandId": "d1b16593-3ea3-4cb4-9cac-53cec0a12cee" }, "button": "left" }, { </pre>
--	---

	<pre> "cmd": "type_text", "meta": { "commandId": "9a991ddb-c019-4e4d- 8e82-15538b2afc0e" }, "text": "123" }, { "cmd": "mouse_move", "meta": { "commandId": "f088de32-257b-4ddc- 98da-e760657baad5" }, "dx": 912, "dy": 739, "smooth": true }, { "cmd": "mouse_click", "meta": { "commandId": "2a1c0a27-8a59-41fd- a02d-a30b550d9962" }, "button": "left" }], "total_commands": 8, "timestamp": "2026-04-21T15:12:41.386542", "execution_time": "7.12s" } </pre>
Status (Pass/Fail)	Pass

6. RESULTS AND DISCUSSIONS

6.1 Results

This section presents the empirical outcomes of implementing and testing the Screen-Aware Task Agent with Gemini-based language reasoning and HID command generation. The goal of this evaluation was to determine whether natural language instructions can be reliably transformed into executable UI actions using structured visual context, while maintaining safety, interpretability, and practical execution quality.

The evaluation was conducted using the test design, where each test case represented a realistic automation scenario. The test set included successful workflows, ambiguous user intent, and missing UI element conditions. This allowed the system to be observed not only under ideal execution conditions, but also under practical uncertainty and failure-prone situations that are common in real interfaces.

6.1.1 Overall Execution Outcomes

Across the implemented test scenarios, the system demonstrated stable end-to-end behavior for structured automation tasks. For tasks such as credential entry, search operations, and deterministic click and type sequences, the model generated coherent rewritten steps and corresponding HID commands with high confidence and consistent structural correctness.

The strongest evidence of system capability was observed in standard deterministic tasks, such as login and search workflows. In these scenarios, the system consistently produced a valid instruction validation result, confirming the feasibility of the task based on the provided visual context. It further generated a coherent sequence of rewritten task steps, effectively decomposing high-level instructions into structured and interpretable actions.

In addition, the system successfully produced structured action-level steps that were accurately aligned with detected user interface elements. These steps were translated

into executable HID command streams, including mouse movement, click actions, text input, and key events. The execution process completed successfully without runtime interruptions, demonstrating the system's ability to perform reliable end-to-end automation in controlled scenarios.

Beyond successful task execution, the system also demonstrated robust protective behavior under constrained conditions. In cases where required interface elements were absent from the visual context, the validation layer correctly identified infeasible instructions and prevented action generation. This indicates that the system is not purely generative but operates with context-aware safety mechanisms that ensure reliable and controlled automation.

Figure 6.1 shows the conversion of generated action steps into executable HID commands in terminal.

```
(venv) PS D:\SLIIT\Y451\RP\Project_works\ScreenAwareTaskAgent> python -m uvicorn llm.api:app --reload --port 8002
INFO: 127.0.0.1:62274 - "GET /llm/health HTTP/1.1" 200 OK
INFO:llm.api: HID generation request: Enter username 'tharushi' in the Username field, enter password '123' in the Pas...
INFO:llm.api: Using Google Gemini API
INFO:llm.api: Gemini API key source: environment
INFO:llm.hid_step_generator: Validating instruction against visual context...
D:\SLIIT\Y451\RP\Project_works\ScreenAwareTaskAgent\venv\lib\site-packages\google\api_core\python_version_support.py:275: FutureWarning: You are using a Python version (3.10.11) which Google will stop supporting in new releases of google.api_core once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, to continue receiving updates for google.api_core past that date.
  warnings.warn(message, FutureWarning)
D:\SLIIT\Y451\RP\Project_works\ScreenAwareTaskAgent\llm\gemini_client.py:45: FutureWarning:
All support for the 'google.generativeai' package has ended. It will no longer be receiving updates or bug fixes. Please switch to the 'google.genai' package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
INFO:llm.gemini_client: Calling Gemini API: model=models/gemini-flash-latest, max_tokens=1500
INFO:llm.gemini_client: Gemini responded in 3.67s
INFO:llm.hid_step_generator: Validation PASSED (confidence: 1.00): All required elements (Username input field, Password input field, and Login button) are present and match the user's instruction.
INFO:llm.hid_step_generator:Stage 1: Generating action plan for: Enter username 'tharushi' in the Username field, e...
INFO:llm.gemini_client: Calling Gemini API: model=models/gemini-flash-latest, max_tokens=1500
INFO:llm.gemini_client: Gemini responded in 9.04s
INFO:llm.hid_step_generator:Stage 1 LLM output:
[
  {"step": 1, "action": "click", "target": "Username field", "x": 863, "y": 475},
  {"step": 2, "action": "type text", "target": "Username field", "text": "tharushi"},
  {"step": 3, "action": "click", "target": "Password field", "x": 863, "y": 583},
  {"step": 4, "action": "type text", "target": "Password field", "text": "123"},
  {"step": 5, "action": "click", "target": "Login button", "x": 912, "y": 739}
]
INFO:llm.hid_step_generator: Stage 1: Generated 5 action steps
INFO:llm.hid_step_generator: Stage 2: Converted 5 actions to 8 HID commands
INFO:llm.hid_step_generator: Three-stage pipeline complete: Validation + 5 actions + 8 HID commands
INFO:llm.api: Generated 5 actions + 8 HID commands in 14.77s
INFO:llm.api: Rewritten Steps:
INFO:llm.api: 1. click: Click Username field
INFO:llm.api: 2. type_text: Type 'tharushi' in Username field
INFO:llm.api: 3. click: Click Password field
INFO:llm.api: 4. type_text: Type '123' in Password field
INFO:llm.api: 5. click: Click Login button
INFO:llm.api: Action Steps (JSON):
INFO:llm.api: Step 1: click - Username field
INFO:llm.api: Step 2: type_text - Username field
INFO:llm.api: Step 3: click - Password field
INFO:llm.api: Step 4: type_text - Password field
INFO:llm.api: Step 5: click - Login button
INFO:llm.api: HID Commands:
INFO:llm.api: mouse_move: {'cmd': 'mouse_move', 'meta': {'commandId': '195be7aa-26c9-42a8-9266-89778910bee1'}, 'dx': 863, 'dy': 475, 'smooth': True}
INFO:llm.api: mouse_click: {'cmd': 'mouse_click', 'meta': {'commandId': 'c53d4ea3-67df-46cc-a42b-e60a1b90c5e0'}, 'button': 'left'}
INFO:llm.api: type_text: {'cmd': 'type_text', 'meta': {'commandId': '339a416b-7026-4ed5-802b-4acfa93b776a'}, 'text': 'tharushi'}
INFO:llm.api: mouse_move: {'cmd': 'mouse_move', 'meta': {'commandId': '55d2ad1d-4242-4995-88a5-dd06718aeebf'}, 'dx': 863, 'dy': 583, 'smooth': True}
INFO:llm.api: mouse_click: {'cmd': 'mouse_click', 'meta': {'commandId': '6db41eb8-beac-4553-ad41-72f45126d1d6'}, 'button': 'left'}
INFO: 127.0.0.1:62276 - "POST /llm/generate_hid HTTP/1.1" 200 OK
```

Figure 6.1: Generated action steps displayed in terminal

Figure 6.2 shows the JSON input sent through Postman to the endpoint, including the user instruction, visual element context (username, password, login), and Gemini configuration parameters for HID step generation.

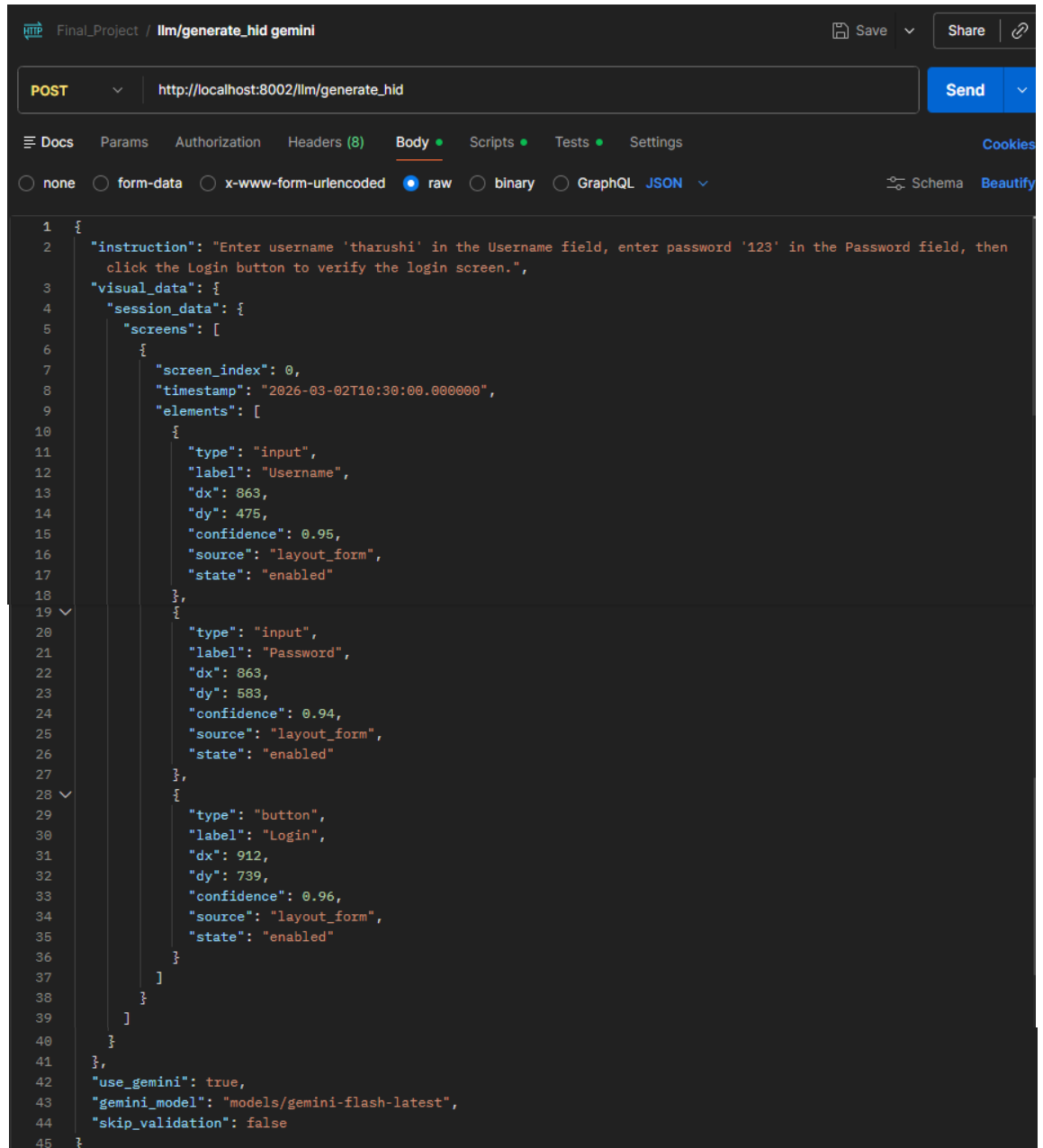


Figure 6.2: Postman request payload

6.1.2 Quantitative Performance Summary

The experimental evaluation revealed a clear variation in execution time and output quality depending on instruction clarity, task complexity, and the richness of the provided visual context. Deterministic instructions with explicitly defined UI targets consistently resulted in lower completion latency and more stable execution outcomes. In contrast, semantically broad or ambiguous instructions required additional reasoning cycles within the LLM, leading to increased latency and variability in output structure.

Analysis of the observed metrics indicates three distinct performance categories. First, low-latency responses were recorded for straightforward visual grounding tasks, where the instruction directly matched clearly identifiable UI elements. In such cases, the model required minimal reasoning and was able to rapidly generate structured action sequences. Second, moderate latency was observed in multi-step workflow tasks, where the system needed to decompose instructions into sequential actions and maintain intermediate context across steps. These tasks introduced additional computational overhead due to step-wise planning and validation.

Third, significantly higher latency was associated with ambiguous or underspecified instructions. In these scenarios, the LLM engaged in deeper semantic reasoning to infer user intent, resolve uncertainties, and construct a plausible execution path. This process often involved implicit re-interpretation of instructions and internal validation, thereby increasing response time. Additionally, in workflows involving multiple interface states or partial visual information, reduced output completeness was observed. Although the system successfully generated API-level responses, some action sequences were incomplete or lacked full contextual grounding.

The observed latency distribution aligns with the inherent characteristics of LLM-based planning systems. Unlike rule-based automation, where execution time is relatively constant, LLM-driven systems exhibit variability due to token-level reasoning processes. The length and complexity of the reasoning path directly influence execution time. When instructions are clear and strongly supported by visual context, the model converges quickly to a solution. However, in cases of ambiguity or weak contextual grounding, the

model allocates additional computational effort to interpret intent, evaluate alternatives, and generate coherent action plans.

Furthermore, these findings highlight a trade-off between reasoning depth and execution efficiency. While deeper reasoning enables the system to handle complex and ambiguous tasks more intelligently, it also introduces latency that may limit real-time applicability. Therefore, optimizing instruction clarity and enhancing prompt engineering strategies can significantly improve performance by reducing unnecessary reasoning overhead and guiding the model toward more efficient decision-making pathways.

6.1.3 Accuracy and Output Structure Quality

A key outcome of this study is not only the successful generation of outputs, but the consistent production of structured and machine-consumable results. The generated responses adhered to predefined schema patterns, including fields for validation status, rewritten task steps, detailed action sequences, command counts, and execution metadata. This consistency is critical for enabling seamless integration with downstream execution components.

The quality of the generated outputs can be analyzed across three dimensions: semantic accuracy, contextual accuracy, and execution readiness.

Semantic Accuracy:

The system demonstrated a strong ability to correctly interpret user intent in deterministic task scenarios. For instance, login-related instructions were reliably translated into logically ordered action sequences, such as focusing on the username field, entering text, shifting focus to the password field, entering credentials, and submitting the form. This indicates that the LLM effectively captures task semantics and converts them into meaningful procedural steps.

Contextual Accuracy:

The generated actions showed a high degree of alignment with the available visual context. The model consistently mapped instructions to UI elements identified in the perception

layer, without generating arbitrary or unsupported coordinates. This confirms that the integration of structured visual summaries enables grounded decision-making and reduces the likelihood of incorrect element selection.

Execution Readiness:

The final output, particularly in the form of HID command sequences, was directly executable and contained sufficient detail for low-level dispatch. Each command included precise parameters such as coordinates, input values, and action types, demonstrating that the system moves beyond descriptive reasoning into practical, execution-ready automation.

These findings are significant because many LLM-based systems are limited to generating explanatory or high-level outputs without ensuring executability. In contrast, this implementation successfully bridges the gap between reasoning and action by producing deterministic, structured command sequences suitable for real-world automation tasks.

6.1.4 Robustness Under Edge Condition

The robustness evaluation highlights the system’s ability to operate reliably under non-ideal and uncertain conditions, which are common in real-world environments. Two primary categories of edge cases were analyzed: missing element conditions and ambiguous instruction scenarios.

Missing Element Conditions:

When required interface elements were absent from the visual context, the system accurately detected infeasibility through its validation layer. Instead of proceeding with uncertain or incorrect actions, it halted execution and reported the missing components. This behavior reduces the risk of erroneous command execution and demonstrates the presence of context-aware safety mechanisms, which are essential for dependable automation.

Ambiguous Instruction Conditions:

The system also exhibited resilience when handling vague or non-technical user instructions, such as “complete the process properly.” In such cases, the LLM was able to

infer a reasonable sequence of actions and generate usable plans. However, this increased robustness came at the cost of higher response latency, as additional reasoning was required to interpret intent and construct appropriate outputs.

A notable contribution of this system is its transparency in both successful and failure scenarios. Even under uncertain conditions, the system provides explicit validation results, confidence scores, and explanatory reasoning rather than failing silently. This level of explainability supports better user trust, facilitates debugging, and enables effective post-execution analysis.

6.1.5 Comparative Behavior Across Scenario Types

A comparative analysis across different scenario categories reveals distinct performance patterns in terms of reliability, latency, and output completeness.

Deterministic Structured Tasks:

These scenarios demonstrated the highest level of reliability and consistency. The system was able to generate complete and well-structured action sequences with minimal latency. The presence of clearly defined instructions and directly identifiable UI elements allowed the model to converge quickly toward accurate and executable plans.

Ambiguous Language Tasks:

Tasks defined using vague or non-specific language showed high eventual success rates, indicating strong semantic reasoning capability. However, these scenarios incurred significantly higher processing time due to additional reasoning required to interpret user intent. This highlights a trade-off between flexibility in instruction handling and computational efficiency.

State-Heavy Sequential Tasks:

Scenarios involving multiple interface states or step dependencies exhibited lower output completeness and reduced confidence levels. In such cases, the system struggled to maintain continuity across transitions, particularly when the visual context did not fully

capture state progression. This resulted in partial plans or simplified action sequences that did not fully represent the intended workflow.

Overall, this categorization highlights an important operational insight: the proposed architecture performs optimally in constrained and well-structured automation tasks, while its effectiveness decreases in long-horizon, state-dependent workflows. This observation suggests the need for enhanced state modeling and context persistence mechanisms to support more complex, sequential task execution in future iterations.

6.1.6 Practical Result Interpretation

From a practical application perspective, the results indicate that the proposed system can function as a reliable automation assistant for repetitive, form-based, and navigation-light tasks. The system demonstrates strong performance under conditions where required interface elements are clearly represented within the visual context, user instructions are explicit and goal-oriented, and interface state transitions are limited and predictable.

Conversely, the system exhibits reduced reliability in more complex scenarios. Performance declines when tasks require extended continuity across multiple interface states, when visual context information is incomplete or fragmented, and when user instructions are broad or insufficiently specified. In such cases, the system may produce partial action plans, lower confidence scores, or increased execution latency due to additional reasoning requirements.

These findings are consistent with the inherent limitations of prompt-based LLM planning systems, where performance is highly dependent on input clarity and contextual grounding. The results further highlight the importance of incorporating explicit state modeling and improved context management mechanisms in future system iterations to enhance performance in long-horizon and dynamic task environments.

6.2 Research Finding

The section presents the key research findings derived from implementation results and experimental evaluation. Each finding is grounded in observed system behavior and supported by empirical evidence from test case execution.

Visual-context-grounded prompting enables effective UI automation:

The integration of structured visual metadata, including element labels, types, and spatial coordinates, with natural language instructions significantly improved the relevance and accuracy of generated actions. The model demonstrated the ability to utilize contextual grounding rather than relying solely on linguistic priors, resulting in more precise and executable action plans.

A validation-first architecture enhances operational safety:

Introducing a validation layer prior to action generation prevented execution in scenarios where required interface elements were missing or conditions were infeasible. This confirms that early-stage guardrails reduce downstream execution risks and improve the overall reliability of the system.

Natural language flexibility can be achieved alongside structured outputs:

The system successfully handled both precise and ambiguous user instructions while consistently producing structured, machine-readable outputs. This demonstrates that user-friendly interaction does not necessarily compromise the determinism required for executable automation.

Confidence scores serve as reliable indicators of execution quality:

High-confidence outputs were consistently associated with complete and coherent action plans, while lower-confidence scores correlated with partial or uncertain outputs. This indicates that confidence metrics can be effectively utilized for adaptive execution control and decision-making in practical deployments.

Latency is strongly influenced by semantic ambiguity:

Execution time increased significantly in scenarios involving ambiguous or underspecified instructions, even when the number of generated actions remained similar. This suggests that latency optimization should focus on improving intent clarity and prompt design rather than solely optimizing the execution pipeline.

Workflow continuity remains a key limitation in complex tasks:

While the system performed reliably in structured, single-state scenarios, it exhibited reduced continuity in long-horizon workflows involving multiple interface transitions. This highlights the need for improved state representation and transition modeling in future system enhancements.

Instruction-to-HID command transformation is technically feasible:

The system successfully demonstrated the end-to-end conversion of natural language instructions into low-level executable HID commands through intermediate planning representations. This confirms the practical viability of the proposed architecture for controlled automation environments.

Transparency mechanisms improve interpretability and debugging:

The explicit inclusion of validation results, confidence scores, rewritten steps, and command metadata enhances system transparency. This level of explainability supports debugging, auditing, and user trust, particularly in academic and enterprise use cases.

The modular architecture supports extensibility:

The separation of components into distinct layers such as API handling, LLM processing, validation, and command generation enables flexible system evolution. Individual modules can be updated or replaced without requiring a complete architectural redesign, supporting long-term maintainability.

6.3 Discussion

This section interprets the research findings in relation to the project objectives, practical implications, and the broader context of AI-driven automation systems.

6.3.1 Interpretation Against Research Objectives

The primary objective of this research was to design and evaluate a screen-aware task execution framework capable of translating natural language instructions into executable user interface interactions. The implementation and experimental results confirm that this objective has been successfully achieved within the scope of the targeted task domain.

The system demonstrated the ability to understand user instructions expressed in natural language, ground these instructions in structured visual context representations, generate sequential action plans, and convert those plans into executable HID command streams. These outcomes provide strong empirical support for the hypothesis that a visually grounded LLM can effectively enable practical UI automation in black-box environments.

6.3.2 Effectiveness of the Proposed Approach

The effectiveness of the architecture can be attributed to its hybrid design, which combines probabilistic reasoning with deterministic constraints. The LLM is responsible for semantic interpretation and plan generation, while the validation and execution layers enforce structural correctness and feasibility.

This integration mitigates two common limitations in existing systems: purely rule-based approaches, which lack flexibility in handling natural language variation, and purely generative approaches, which may produce ungrounded or unsafe outputs. By introducing structured validation and machine-readable output formats, the system achieves a balance between adaptability and control.

6.3.3 Implication of Human-Computer Interaction

From a human-computer interaction perspective, this research demonstrates a practical pathway toward low-code or no-code automation systems. Users can express goals using natural language, and the system translates these goals into executable interaction sequences without requiring manual scripting.

This has significant implications for accessibility-focused automation, productivity enhancement in enterprise workflows, and human-in-the-loop systems where users supervise execution rather than encode it. The findings suggest that natural-language-driven automation is no longer purely conceptual but can be effectively operationalized with appropriate grounding and validation mechanisms.

6.3.4 Reliability-Latency Trade-off

A key observation from the results is the trade-off between semantic robustness and execution latency. While the system successfully handles ambiguous instructions, this capability introduces additional reasoning overhead, resulting in increased response time.

This presents an important design consideration for deployment. Systems may prioritize either accuracy and flexibility, accepting higher latency, or efficiency, by enforcing stricter input constraints. A balanced approach could involve adaptive execution strategies, such as fast-path processing for explicit instructions and clarification mechanisms for ambiguous inputs. This would maintain usability while reducing worst-case latency.

6.3.5 Workflow Continuity as a Limitation and Opportunity

The reduced performance observed in long-horizon, state-dependent workflows should be interpreted as a targeted architectural limitation rather than a fundamental failure. While the current system performs well in localized interaction planning, it lacks robust mechanisms for tracking state transitions across extended workflows.

Future enhancements should focus on incorporating explicit state representations, transition-aware memory, and goal progression tracking. Introducing planner-executor separation and checkpointing mechanisms could further improve continuity and reliability in complex task sequences.

6.3.6 Trust, Safety and Explainability

The inclusion of validation outputs, confidence scores, and structured reasoning contributes significantly to system transparency and trustworthiness. Unlike opaque automation systems, this architecture provides clear signals regarding execution feasibility and reliability.

In practical deployment, this enables the implementation of safety-aware strategies such as confidence-based execution thresholds, user confirmation for uncertain actions, and detailed logging for audit and recovery. These features are critical for ensuring safe adoption in environments where incorrect automation may have significant consequences.

6.3.7 Threats to Validity

The findings of this study are subject to several limitations. The evaluation was conducted on a limited set of scenarios, and performance may vary across broader application domains. The system also depends on the accuracy of visual perception, where detection errors can affect planning outcomes. Additionally, results are influenced by the chosen LLM configuration, and performance may change with different models. Finally, sensitivity to instruction phrasing can impact both accuracy and execution time.

6.3.8 Practical Deployment Implications

Based on the observed results, the system is suitable for deployment in constrained automation scenarios such as structured form entry, account management workflows, and guided navigation tasks. Effective deployment should incorporate confidence-based execution control, clear user feedback mechanisms, and logging capabilities for traceability.

Additionally, the use of standardized prompt templates can help reduce ambiguity and improve efficiency, enabling more predictable system behavior in real-world applications.

7. CONCLUSIONS

This research set out to design and validate a Screen-Aware Task Agent capable of translating natural language instructions into executable user-interface actions through visual-context grounding and Gemini-based reasoning. The overall objective was to determine whether a language-driven automation pipeline could provide both practical usability and reliable execution in realistic interaction scenarios. Based on the implementation and evaluation results, this objective was achieved.

The developed system successfully demonstrated a complete end-to-end workflow. User instructions were interpreted semantically, validated against structured visual context, converted into high-level action steps, and then transformed into low-level HID commands for execution. This confirms that the proposed architecture is not only conceptually valid but also operationally implementable for real task automation settings. The experimental outcomes showed that the agent can generate coherent, structured, and executable outputs for common workflows such as credential entry, search, and sequential interaction tasks.

A key contribution of this work is the integration of a validation-first mechanism prior to action execution. Rather than relying on unconstrained generation, the system checks whether required interface elements are present and whether the requested action is feasible in the given context. This design decision significantly improves reliability and safety. In infeasible scenarios, the agent reports validation outcomes instead of proceeding with potentially incorrect actions, which is essential for trustworthy automation in practice.

Another important conclusion is that natural language flexibility can be preserved without losing structured output quality. Even when instructions were less explicit, the system was able to produce usable task plans and command sequences. This indicates that non-technical users can interact with the agent using intuitive language while the backend maintains machine-consumable, deterministic formats for execution. At the same time, the experiments showed that ambiguous requests increase inference time, highlighting a practical tradeoff between semantic robustness and runtime efficiency.

From a research perspective, the study confirms the central hypothesis that visually grounded LLM reasoning can drive dependable UI automation under practical constraints. The architecture’s modular decomposition into API handling, context validation, reasoning, and command generation also provides strong extensibility for future improvements. This modularity supports maintainability and enables component-level optimization without redesigning the entire system.

Despite these strengths, the work also identifies clear improvement areas. Performance variability under semantically uncertain instructions indicates the need for better intent normalization, prompt optimization, and response-time engineering. In addition, task continuity across more complex state transitions remains a priority direction for future development. These limitations do not weaken the core contribution; rather, they define a focused roadmap for the next stage of the project.

In summary, this research contributes a practical and technically validated foundation for natural-language-driven assistive automation. The implemented Screen-Aware Task Agent demonstrates that combining language intelligence with context-aware validation can produce executable, interpretable, and safer automation behavior. Future work can build directly on this foundation to improve speed, workflow continuity, and broader deployment readiness across diverse interface environments.

8. REFERENCES

- [1] M. Harman, P. McMinn, J. T. de Souza, and S. Yoo, "Search based software engineering: Techniques, taxonomy, tutorial," Empirical Software Engineering, 2012.
- [2] A. Mesbah, A. van Deursen, and S. Lenseslink, "Crawling Ajax-based web applications through dynamic analysis of user interface state changes," ACM Transactions on the Web, 2012.
- [3] R. Smith, "An overview of the Tesseract OCR engine," Proc. ICDAR, 2007.
- [4] T. Brown et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [5] S. Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2015.
- [6] D. Amershi et al., "Software Engineering for Machine Learning: A Case Study," ICSE, 2019.
- [7] S. M. M. K. Rahman and A. M. Memon, "The forgotten GUI testing: A systematic mapping study," in 2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2018, pp. 1-10.
- [8] W3C, "WebDriver," W3C Recommendation, Jun. 2018. [Online]. Available: <https://www.w3.org/TR/webdriver/>
- [9] S. R. Choudhary, H. Versee, and A. Orso, "WEBDIFF: Automated identification of cross-browser issues in web applications," in 2010 IEEE International Conference on Software Maintenance, 2010, pp. 1-10.
- [10] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629-633.
- [11] B. Deka, Z. Huang, and R. Kumar, "Rico: A mobile app dataset for building data-driven design applications," in Proc. 30th Annual ACM Symposium on User Interface Software and Technology (UIST), 2017, pp. 845-854.
- [12] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 1877-1901.

- [13] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824-24837.
- [14] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [15] S. Zhou et al., “WebArena: A realistic web environment for building autonomous agents,” *arXiv:2307.13854*, 2023.
- [16] USB Implementers Forum, “Device Class Definition for Human Interface Devices (HID),” Version 1.11, 2001.
- [17] Espressif Systems, *ESP32 Technical Reference Manual*, Espressif Systems, 2023.
- [18] Yunpeng Song, Y. B. Y. T. G. M. Z. C. (n.d.). *VisionTasker: Mobile Task Automation Using Vision Based UI Understanding and LLM Task Planning*.
- [19] Panneer Selvam Viswanathan. (n.d.). *Prompt Engineering for Conversational AI Systems: A Systematic Review of Techniques and Applications*.
- [20] gaurav gupta. (n.d.). *Vision-Language-Action (VLA) Models: LLMs for robots*.
- [21] Rupesh Garg. (2025). *How to Choose the Best LLM Tools for Your Test Automation Strategy*.
- [22] Wang, L., Chen, X., Deng, X., Wen, H., You, M., Liu, W., Li, Q., & Li, J. (2024). Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs. *Npj Digital Medicine*, 7(1), 41. <https://doi.org/10.1038/s41746-024-01029-4>
- [23] S, J. P. B., D, B. K., A, P. S., & D, M. G. (2025). *Prompt Engineering for Large Language Models: A Systematic Review and Future Directions*. <https://doi.org/10.21203/rs.3.rs-6551106/v1>
- [24] Marvin, G., Hellen, N., Jjingo, D., & Nakatumba-Nabende, J. (2024). *Prompt Engineering in Large Language Models* (pp. 387–402). https://doi.org/10.1007/978-981-99-7962-2_30
- [25] Pawlik, L. (2025). *How the Choice of LLM and Prompt Engineering Affects Chatbot Effectiveness*. *Electronics*, 14(5), 888. <https://doi.org/10.3390/electronics14050888>

9. APPENDICES

Appendix - A: Plagiarism report



Class Portfolio

NOW VIEWING: [HOME](#) > [4TH YEAR IT 2ND SEMESTER 2026](#) > 4TH YEAR IT 2ND SEM ASSIGNMENT



Submit work
4th Year IT 2nd sem Assignment ▾


Help

Your work

Resubmit

Title	Submitted	Grade	Similarity	Feedback	More
IT22083296- Draft Final Report	Apr 22, 2026 1:24 AM	-- / 100	5%		